

Universidade Federal de Alfenas

Instituto de Ciências Exatas

Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Diferenciação e Integração Numéricas:
Análise, Comparação e Aplicações em Python

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Modalidade: Atividade de investigação em dupla

Entrega: Relatório .pdf + código-fonte .py

Apresentação

Este plano investiga numericamente dois problemas fundamentais do Cálculo: **derivar** e **integrar**. Quando a expressão analítica não está disponível — porque os dados vêm de experimentos, simulações ou funções sem primitiva elementar — métodos numéricos são a única opção.

O foco desta atividade não é apenas obter a resposta correta: é **compreender o comportamento do erro**, saber *quando* cada método é mais adequado, e desenvolver a capacidade de **analisar quantitativamente** os resultados.

Objetivos

Ao concluir esta investigação, o aluno será capaz de:

- a) Implementar as fórmulas de diferenças finitas progressiva, regressiva e central para f' e f'' ;
- b) Identificar e explicar o conflito entre erro de truncamento e erro de arredondamento;
- c) Determinar experimentalmente o passo ótimo h_{opt} de cada fórmula;
- d) Aplicar as regras do Ponto Médio, Trapézios e Simpson sobre funções analíticas e dados tabelados;
- e) Estimar o erro de integração e determinar o número mínimo de subintervalos para precisão garantida;
- f) Aplicar a Quadratura Gaussiana e comparar sua eficiência com Newton-Cotes;
- g) Integrar derivação e integração numéricas em problemas aplicados realistas.

Atenção

O objetivo **não** é apenas executar o código. É **observar, medir, comparar e interpretar** os resultados à luz da teoria. Cada questão pede uma análise — não apenas um número.

Sumário

Apresentação	1
1 Fórmulas de Diferenças Finitas	3
1.1 Verificação e primeiros experimentos	4
1.2 Conflito entre truncamento e arredondamento	5
1.3 Diferenciação de tabela experimental	5
1.4 Segunda derivada e aplicações	5
2 Newton-Cotes: Trapézios e Simpson	6
2.1 Verificação das fórmulas	7
2.2 Análise de erro e determinação de n	7
2.3 Integração de dados experimentais	8
3 Quadratura Gaussiana e Comparação de Métodos	9
3.1 Eficiência da Quadratura Gaussiana	10
3.2 Limitação dos nós de Chebyshev	10
4 Análise Comparativa Global	11
5 Projeto Integrador: Cálculo Variacional e Física	12
5.1 Comprimento de curva	12
5.2 Trabalho realizado por uma força variável	12
5.3 Distribuição Normal e Probabilidade	13
6 Desafio (Opcional — Pontuação Extra)	14
Entrega e Critérios de Avaliação	16
A Esqueleto do Script Principal	17
B Referências	18

1. Fórmulas de Diferenças Finitas

Dica

Salve o bloco abaixo em `diferencas.py`. Não o modifique antes de responder as questões desta seção.

```

1 import numpy as np
2
3 #           Formulas de dois pontos (ordem 1)
4
5 def df_progressiva(f, x, h):
6     """Diferença progressiva:  $[f(x+h) - f(x)] / h$        $O(h)$ ."""
7     return (f(x + h) - f(x)) / h
8
9 def df_regressiva(f, x, h):
10    """Diferença regressiva:  $[f(x) - f(x-h)] / h$        $O(h)$ ."""
11    return (f(x) - f(x - h)) / h
12
13 def df_central(f, x, h):
14    """Diferença central:  $[f(x+h) - f(x-h)] / (2h)$        $O(h^2)$ ."""
15    return (f(x + h) - f(x - h)) / (2 * h)
16
17 #           Formulas de três pontos (ordem 2)
18
19 def df_prog3(f, x, h):
20    """Progressiva 3 pts:  $[-3f(x) + 4f(x+h) - f(x+2h)] / (2h)$ 
21     $O(h^2)$ ."""
22    return (-3*f(x) + 4*f(x + h) - f(x + 2*h)) / (2 * h)
23
24 def df_retro3(f, x, h):
25    """Retroativa 3 pts:  $[f(x-2h) - 4f(x-h) + 3f(x)] / (2h)$ 
26     $O(h^2)$ ."""
27    return (f(x - 2*h) - 4*f(x - h) + 3*f(x)) / (2 * h)
28
29 #           Segunda derivada (ordem 2)
30
31 def d2f_central(f, x, h):
32    """Segunda derivada central:  $[f(x-h) - 2f(x) + f(x+h)] / h^2$ 
33     $O(h^2)$ ."""
34    return (f(x - h) - 2*f(x) + f(x + h)) / h**2
35
36 #           Derivada de tabela de pontos
37
38 def derivada_tabela(xi, fi):
39    """

```

```

33     Calcula f'(x_k) para todos os pontos de uma tabela.
34     Interior: formula central (O(h^2)).
35     Bordas: formulas de 3 pontos de 2a ordem.
36     """
37     xi = np.asarray(xi, dtype=float)
38     fi = np.asarray(fi, dtype=float)
39     n = len(xi)
40     h = xi[1] - xi[0]      # espacamento uniforme
41     df = np.zeros(n)
42     # Borda esquerda: progressiva de 2a ordem
43     df[0] = (-3*fi[0] + 4*fi[1] - fi[2]) / (2 * h)
44     # Interior: central de 3 pontos
45     df[1:-1] = (fi[2:] - fi[:-2]) / (2 * h)
46     # Borda direita: retroativa de 2a ordem
47     df[-1] = (fi[-3] - 4*fi[-2] + 3*fi[-1]) / (2 * h)
48     return df

```

Listing 1: diferencas.py — fórmulas de diferenças finitas

1.1. Verificação e primeiros experimentos

Q1.1. Para $f(x) = \sin x$ em $x_0 = \pi/3$, o valor exato é $f'(\pi/3) = \cos(\pi/3) = 0,500000$. Use as três fórmulas de **dois pontos** com $h = 0,1$. Para cada fórmula, calcule o valor aproximado e o erro absoluto.

(a) Preencha a tabela abaixo:

Fórmula	Valor aproximado	Erro $ e $	Ordem esperada
Progressiva			$\mathcal{O}(h)$
Regressiva			$\mathcal{O}(h)$
Central			$\mathcal{O}(h^2)$

(b) Repita com $h = 0,01$. Os erros diminuíram pelo fator esperado (2 para as de 1ª ordem, 100 para a central)?

(c) Por que a fórmula central tem erro $\mathcal{O}(h^2)$ mas usa apenas dois pontos ao redor de x_0 , assim como a progressiva?

Q1.2. Para a mesma função, use agora as fórmulas de **três pontos** com $h = 0,1$ e $h = 0,01$.

(a) Calcule `df_prog3`, `df_retro3` e `df_central`. Monte uma tabela com os erros.

(b) Ao reduzir h de 0,1 para 0,01 (fator 10), por quanto o erro de cada fórmula foi reduzido? Isso é consistente com erro $\mathcal{O}(h^2)$?

(c) Por que a fórmula central de 3 pontos produz **o mesmo resultado** que a central de 2 pontos?

1.2. Conflito entre truncamento e arredondamento

Q1.3. Para $f(x) = e^x$ em $x_0 = 1,0$ (valor exato: $f'(1,0) = e \approx 2,71828$), use a fórmula `df_central` com $h = 10^{-1}, 10^{-2}, \dots, 10^{-16}$ (`np.logspace(0, -16, 200)`).

- Plote o erro absoluto em escala log-log. Descreva o que acontece à medida que h decresce: o erro primeiro cai, depois *sobe*. Por que?
- Identifique experimentalmente o h_{opt} (ponto de mínimo do erro). Qual é seu valor aproximado? Compare com a estimativa teórica $h_{opt} \approx \varepsilon_m^{1/3} \approx 10^{-5}$ para a fórmula central.
- Repita para `df_progressiva`. O h_{opt} é diferente? Compare com $h_{opt} \approx \sqrt{\varepsilon_m} \approx 10^{-8}$.
- Qual fórmula atinge menor erro mínimo: progressiva ou central? Por quanto? Isso é consistente com as ordens $\mathcal{O}(h)$ e $\mathcal{O}(h^2)$?

1.3. Diferenciação de tabela experimental

Q1.4. A tabela abaixo registra a posição (em metros) de um veículo em função do tempo (em segundos), coletada por GPS com resolução de 0,1 m:

t (s)	0,0	0,5	1,0	1,5	2,0	2,5	3,0	3,5
x (m)	0,0	1,2	4,8	10,6	18,5	27,1	35,8	43,6

- Use `derivada_tabela` para estimar a velocidade $v(t) = \dot{x}(t)$ em cada instante.
- Use `d2f_central` (adaptada para tabela) para estimar a aceleração $a(t) = \ddot{x}(t)$ nos pontos interiores.
- O veículo freia em algum instante? Em qual ponto a aceleração muda de sinal?
- Sabendo que os dados têm erro de $\pm 0,1$ m, qual é a incerteza na velocidade estimada? (*Dica:* para a fórmula central, $\delta v \approx 2 \times 0,1/(2h) = 0,2/h$. Calcule para $h = 0,5$ s.)
- Discuta: com dados experimentais ruidosos, faz sentido usar h muito pequeno? Por quê?

1.4. Segunda derivada e aplicações

Q1.5. Para $f(x) = \ln x$ em $x_0 = 2,0$:

- O valor exato é $f''(2,0) = -1/4 = -0,2500$. Estime com `d2f_central` usando $h = 0,1$ (use $f(1,9) = \ln(1,9)$, $f(2,0) = \ln(2,0)$, $f(2,1) = \ln(2,1)$).
- Calcule o erro. A cota teórica é $|E| \leq \frac{h^2}{12} |f^{(4)}(\xi)|$. Para $f(x) = \ln x$, tem-se $f^{(4)}(x) = 6/x^4$. Com $\xi \in [1,9, 2,1]$: calcule a cota e compare com o erro real.
- Repita com $h = 0,05$ e $h = 0,01$. Verifique que o erro reduz por fator 4 a cada vez que h é reduzido à metade.

2. Newton-Cotes: Trapézios e Simpson

Dica

Salve o bloco abaixo em `integracao.py`.

```

1 import numpy as np
2
3 def ponto_medio(f, a, b, n):
4     """Regra do Ponto Médio composta       $O(h^2)$ ."""
5     h = (b - a) / n
6     xi = np.linspace(a + h/2, b - h/2, n)
7     return h * np.sum(f(xi))
8
9 def trapezios(f, a, b, n):
10    """Regra dos Trapézios composta       $O(h^2)$ .
11    Coeficientes: 1, 2, 2, ..., 2, 1."""
12    xi = np.linspace(a, b, n + 1)
13    fi = f(xi)
14    h = (b - a) / n
15    return h / 2 * (fi[0] + 2 * np.sum(fi[1:-1]) + fi[-1])
16
17 def simpson13(f, a, b, n):
18    """Regra de Simpson 1/3 composta       $O(h^4)$ . n deve ser par.
19    Coeficientes: 1, 4, 2, 4, ..., 4, 1."""
20    if n % 2 != 0:
21        raise ValueError("n deve ser par para Simpson 1/3")
22    xi = np.linspace(a, b, n + 1)
23    fi = f(xi)
24    h = (b - a) / n
25    return h / 3 * (fi[0]
26                    + 4 * np.sum(fi[1:-1:2])
27                    + 2 * np.sum(fi[2:-2:2])
28                    + fi[-1])
29
30 def trapezios_tabela(xi, fi):
31    """Trapézios para dados tabelados (espaçamento n o
32    precisa ser uniforme)."""
33    xi, fi = np.asarray(xi, float), np.asarray(fi, float)
34    return np.sum((fi[:-1] + fi[1:]) * np.diff(xi) / 2)
35
36 def simpson13_tabela(xi, fi):
37    """Simpson 1/3 para tabela      n deve ser par."""
38    n = len(fi) - 1
39    if n % 2 != 0:
40        raise ValueError("n deve ser par")
41    h = (xi[-1] - xi[0]) / n
42    return h / 3 * (fi[0]
43                    + 4 * np.sum(fi[1:-1:2])
44                    + 2 * np.sum(fi[2:-2:2])
45                    + fi[-1])

```

```

45
46 def cota_trapezio(f2max, a, b, n):
47     """Cota superior do erro dos Trapézios compostos."""
48     h = (b - a) / n
49     return f2max * (b - a)**3 / (12 * n**2)
50
51 def cota_simpson(f4max, a, b, n):
52     """Cota superior do erro de Simpson 1/3 compostos."""
53     return f4max * (b - a)**5 / (180 * n**4)

```

Listing 2: integracao.py — regras de Newton-Cotes

2.1. Verificação das fórmulas

Q2.1. Calcule $\int_0^1 x^2 dx = \frac{1}{3}$ com as três regras (Ponto Médio, Trapézios e Simpson) usando $n = 4$.

- Antes de calcular: qual método você espera que seja exato? Por quê? (*Dica:* pense no grau do polinômio e na precisão de cada regra.)
- Após calcular: seus resultados confirmam a expectativa? Mostre os valores numéricos.
- Repita para $\int_0^1 x^3 dx = \frac{1}{4}$. Agora Simpson ainda é exato? Por quê?

Q2.2. Calcule $\int_1^2 \frac{1}{x} dx = \ln 2 \approx 0,693147$ com Trapézios e Simpson usando $n = 4, 8, 16$.

- Preencha a tabela comparativa:

n	Trapézios		Simpson 1/3	
	Resultado	Erro	Resultado	Erro
4				
8				
16				

- Ao dobrar n : por qual fator o erro dos Trapézios foi dividido? E o de Simpson? Isso é consistente com as ordens $\mathcal{O}(n^{-2})$ e $\mathcal{O}(n^{-4})$?
- Com $n = 4$, quantas vezes Simpson é mais preciso que Trapézios? Ambos usam 5 pontos de avaliação — o que torna Simpson melhor?

2.2. Análise de erro e determinação de n

Q2.3. Use as cotas de erro para determinar o número mínimo de subintervalos n que garante precisão de 10^{-5} nas integrais abaixo. Para cada uma, calcule n teoricamente e depois verifique computacionalmente qual é o menor n que realmente atinge o erro pedido.

- (a) $\int_0^\pi \sin x \, dx = 2$, pela Regra dos Trapézios.
(Dica: $f''(x) = -\sin x$, $\max_{[0,\pi]} |f''| = 1$.)
- (b) $\int_0^1 e^x \, dx = e - 1$, pela Regra de Simpson 1/3.
(Dica: $f^{(4)}(x) = e^x$, $\max_{[0,1]} |f^{(4)}| = e$.)

Houve discrepância entre n teórico e experimental? O que isso indica sobre a qualidade das cotas?

Q2.4. Para $\int_0^\pi x^2 \sin x \, dx = \pi^2 - 4 \approx 5,870$, use Trapézios e Simpson com $n = 4, 8, 16, 32$.

- (a) Plote o erro em escala log-log (eixo x : $\log n$, eixo y : $\log |E|$). As retas têm inclinações próximas de -2 e -4 ?
- (b) Ao dobrar n : o erro de Trapézios divide por ≈ 4 ? O de Simpson divide por ≈ 16 ? Verifique numericamente.
- (c) Com qual n Simpson supera Trapézios com $n = 100$ em termos de precisão? Quantas avaliações de f cada um usa nesse ponto?

2.3. Integração de dados experimentais

Por que integrar dados é diferente de integrar funções?

Quando temos dados experimentais (e não uma fórmula analítica), não podemos refinar n arbitrariamente — temos apenas os pontos disponíveis. A escolha do método e a análise do erro ficam mais delicadas.

Q2.5. A potência elétrica $P(t)$ (em watts) de um circuito experimental foi medida a cada 0,5 s:

t (s)	0,0	0,5	1,0	1,5	2,0	2,5	3,0	3,5	4,0	4,5
P (W)	0,0	12,4	22,8	30,1	34,2	35,0	32,8	27,5	20,3	12,1

A energia total dissipada é $E = \int_0^{4,5} P(t) \, dt$ (em joules).

- (a) Use `trapezios_tabela` para estimar E .
- (b) Use `simpson13_tabela` (verifique: $n = 9$ é ímpar! Como proceder?). (Sugestão: use Simpson nos primeiros 8 subintervalos e Trapézio no último, ou explique por que isso é válido.)
- (c) A diferença entre os dois resultados dá uma ideia do erro. Qual estimativa você consideraria mais confiável? Por quê?
- (d) Se a precisão exigida for $\pm 0,1$ J e os dados têm erro de $\pm 0,5$ W, a precisão é limitada pelo método numérico ou pela incerteza dos dados?

3. Quadratura Gaussiana e Comparação de Métodos

Dica

Adicione o bloco abaixo ao arquivo `integracao.py`.

```

1 from numpy.polynomial.legendre import leggauss
2
3 def gauss_legendre(f, a, b, n):
4     """
5     Quadratura de Gauss-Legendre com n pontos.
6     Exata para polinômios de grau <= 2n-1.
7     """
8     t, A = leggauss(n)                # n s e pesos
9     x = (b - a) / 2 * t + (b + a) / 2  # transf. para [a
10    , b]
11    return (b - a) / 2 * np.sum(A * f(x))
12
13 def tabela_comparativa(f, a, b, exato, ns_nc, ns_gauss):
14     """
15     Gera tabela comparando Newton-Cotes e Gauss-Legendre.
16     ns_nc: lista de n para Trapezios e Simpson.
17     ns_gauss: lista de n para Gauss-Legendre.
18     """
19     print(f"{'M todo ':25s} {'n':>5s} {'Resultado':>14s} {'Erro'
20     ':>12s}")
21     print("-" * 60)
22     for n in ns_nc:
23         T = trapezios(f, a, b, n)
24         print(f"{'Trap zios ':25s} {n:>5d} {T:>14.8f} {abs(T-
25         exato):>12.2e}")
26     for n in ns_nc:
27         if n % 2 == 0:
28             S = simpson13(f, a, b, n)
29             print(f"{'Simpson 1/3':25s} {n:>5d} {S:>14.8f} {abs(
30             S-exato):>12.2e}")
31     for n in ns_gauss:
32         G = gauss_legendre(f, a, b, n)
33         print(f"{'Gauss-Legendre':25s} {n:>5d} {G:>14.8f} {abs(G
34         -exato):>12.2e}")

```

Listing 3: Adição ao `integracao.py` — Quadratura Gaussiana

3.1. Eficiência da Quadratura Gaussiana

A pergunta central

Newton-Cotes usa nós *fixados* e equiespaçados. A Quadratura Gaussiana usa nós *otimizados*. Com o mesmo número de avaliações de f , qual método é mais preciso?

Q3.1. Para $\int_1^2 \frac{1}{x} dx = \ln 2 \approx 0,693147$, use `tabela_comparativa` com:

- Newton-Cotes: $n = 2, 4, 8, 16, 32$;
 - Gauss-Legendre: $n = 2, 3, 4, 5, 6$.
- (a) Gauss-Legendre com $n = 2$ (2 avaliações de f) é mais preciso que Simpson com $n = 4$ (5 avaliações)? Explique.
- (b) Com quantos pontos de Gauss-Legendre você consegue superar Simpson com $n = 32$ (33 avaliações)?
- (c) Por que Gauss-Legendre converge mais rápido? (*Dica:* pense no grau máximo do polinômio que cada método integra exatamente.)

Q3.2. A função $f(x) = e^{-x^2}$ não tem primitiva elementar. Para estimar $\int_0^1 e^{-x^2} dx \approx 0,746824$ (referência: `scipy.special.erf(1) * np.sqrt(np.pi)/2`):

- (a) Aplique Trapézios, Simpson e Gauss-Legendre com $n = 2, 4, 8, 16, 32$ (apenas os valores de n aplicáveis).
- (b) Plote o erro de cada método vs. número de avaliações de f em escala log-log. Qual método tem a melhor curva “precisão por custo”?
- (c) Para obter erro $< 10^{-8}$: qual método precisa de menos avaliações de f ? Estime quantas.
- (d) Por que o gráfico de Gauss-Legendre não é uma reta no plano log-log (diferente de Trapézios e Simpson)?

3.2. Limitação dos nós de Chebyshev

Q3.3. A Quadratura Gaussiana é ótima para funções analíticas suaves. Mas o que acontece com funções que apresentam comportamento irregular?

Para $f(x) = |x - 0,5|$ em $[0, 1]$ (valor exato = 0,25):

- (a) Calcule o erro de Trapézios, Simpson e Gauss-Legendre com $n = 2, 4, 8, 16$.
- (b) Qual método converge mais rápido para esta função? Isso contradiz os resultados da questão anterior?
- (c) Explique o comportamento de Gauss-Legendre: por que a taxa de convergência cai para funções não-suaves?
- (d) O que esse experimento nos ensina sobre a escolha do método em situações práticas?

4. Análise Comparativa Global

Síntese da investigação

Esta seção não introduz código novo: usa tudo que foi construído. O objetivo é desenvolver **julgamento numérico** — a capacidade de escolher o método certo para cada situação.

Q4.1. Preencha a tabela de síntese com base nos experimentos das seções anteriores:

Aspecto	Trapézios	Simpson 1/3	Gauss- Legendre	Dif. Central
Erro global				
Avals. de f (geral)				
Exato para grau				
Requer n especial?				
Funciona com tabela?				
Melhor cenário				
Pior cenário				

Q4.2. Três estudantes propõem abordagens para calcular $\int_0^{100} f(t) dt$, onde $f(t)$ é a tensão de um sinal elétrico amostrado a cada 0,5 s (total de 201 pontos).

- **Estudante A:** Trapézios com todos os 201 pontos.
- **Estudante B:** Simpson 1/3 com todos os 200 subintervalos (par).
- **Estudante C:** Gauss-Legendre com $n = 5$ no intervalo todo.

- Qual proposta é viável? Qual tem um problema conceitual? Explique.
- Qual você recomenda para dados experimentais? Justifique quantitativamente.
- Se o sinal tivesse descontinuidades (chaveamento elétrico), sua recomendação mudaria?

5. Projeto Integrador: Cálculo Variacional e Física

Motivação. Nesta seção, derivação e integração numéricas são usadas juntas em problemas realistas. Os dados são gerados numericamente para que o erro possa ser verificado — mas os métodos são os mesmos usados em problemas experimentais reais.

5.1. Comprimento de curva

O comprimento de arco de $y = f(x)$ em $[a, b]$ é:

$$L = \int_a^b \sqrt{1 + [f'(x)]^2} dx.$$

O integrando depende de $f'(x)$ — que pode precisar ser calculado numericamente.

Q5.1. Para a parábola $f(x) = x^2$ em $[0, 2]$, o comprimento exato é $L = \sqrt{5} + \frac{1}{2} \ln(2 + \sqrt{5}) \approx 4,6468$.

- Usando **derivação analítica** ($f'(x) = 2x$), calcule L por Simpson 1/3 com $n = 10$. Qual é o erro?
- Agora, usando apenas os valores de f nos pontos da malha (sem a fórmula de f'), estime $f'(x_k)$ pela fórmula central e use esses valores para calcular L . Compare com o resultado anterior.
- O erro do passo (b) é maior que o do (a)? A diferenciação numérica introduz erro adicional?
- Repita com $n = 20$ e $n = 40$. Plote o erro vs. n . Qual é a ordem de convergência observada?

5.2. Trabalho realizado por uma força variável

O trabalho realizado por uma força $F(x)$ ao mover um objeto de $x = a$ até $x = b$ é $W = \int_a^b F(x) dx$.

Q5.2. Os dados experimentais abaixo representam a força (em N) aplicada a um objeto ao longo de um trilho de 6 m:

x (m)	0	0,5	1,0	1,5	2,0	2,5	3,0	3,5	4,0	4,5	5,0	5,5	6,0
F (N)	0	8	15	20	24	26	27	26	22	17	11	5	0

- Calcule W por Trapézios e Simpson 1/3.
- Determine em qual ponto a força é máxima usando `derivada_tabela` para encontrar onde $F'(x) \approx 0$.
- A força cresce ou decresce mais rapidamente? Em qual região a aceleração (variação da força) é maior em magnitude?
- Se a massa do objeto é 2 kg e ele parte do repouso, estime sua velocidade ao final do percurso usando $W = \Delta KE = \frac{1}{2}mv^2$.

5.3. Distribuição Normal e Probabilidade

A função de distribuição acumulada (FDA) normal não tem primitiva elementar:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-t^2/2} dt \approx \frac{1}{\sqrt{2\pi}} \int_{-6}^x e^{-t^2/2} dt.$$

Q5.3. Calcule $\Phi(1,96)$, $\Phi(2,58)$ e $\Phi(3,00)$ numericamente, comparando com os valores tabelados: 0,9750, 0,9951 e 0,9987.

- (a) Use Simpson 1/3 com $n = 100$ no intervalo $[-6, x]$ para cada valor.
- (b) Use Gauss-Legendre com $n = 5, 10, 20$ no mesmo intervalo. Com qual n Gauss-Legendre supera Simpson com $n = 100$?
- (c) A cauda da distribuição (x grande) torna a integração mais difícil? Por que truncar em -6 (e não em $-\infty$) é adequado?
- (d) **Aplicação:** Em um teste de hipótese com $\alpha = 5\%$ (bicaudal), o valor crítico é $z_{\alpha/2}$ tal que $\Phi(z_{\alpha/2}) = 0,975$. Use a **interpolação inversa** (implementada no capítulo de Interpolação) sobre os valores de Φ calculados para estimar $z_{0,025}$. Compare com o valor exato $\approx 1,96$.

6. Desafio (Opcional — Pontuação Extra)

Desafio Computacional

As questões desta seção são opcionais e destinadas a alunos que desejam aprofundar além do exigido. Requerem criatividade na implementação e análise.

Q6.1. Richardson Extrapolation. Dado que o erro dos Trapézios é $E_T(h) = c_2 h^2 + c_4 h^4 + \dots$, é possível combinar duas aproximações $T(h)$ e $T(h/2)$ para cancelar o termo h^2 e obter um método de ordem 4:

$$R(h) = \frac{4T(h/2) - T(h)}{3}.$$

Esta é a **Regra de Romberg** (primeiro passo).

- Implemente $R(h)$ e verifique que ele é equivalente a Simpson 1/3.
- Aplique o processo iterativamente (eliminando sucessivamente h^4 , h^6 , etc.) para $\int_1^2 1/x \, dx$ com $h = 0,5, 0,25, 0,125$. Construa a tabela de Romberg. Com quantas avaliações de f você atinge erro $< 10^{-10}$?
- Compare a eficiência (erro por avaliação de f) com Gauss-Legendre.

Q6.2. Integração de Monte Carlo. Para integrais em alta dimensão, Monte Carlo é o método padrão:

$$\int_{\Omega} f(x) \, d\Omega \approx V(\Omega) \cdot \frac{1}{N} \sum_{i=1}^N f(x_i), \quad x_i \sim \text{Uniforme}(\Omega).$$

- Implemente Monte Carlo para $\int_0^1 e^{-x^2} \, dx$ com $N = 10^2, 10^3, 10^4, 10^5$. Calcule o erro médio (média de 20 execuções).
- O erro decai como $\mathcal{O}(N^{-1/2})$. Verifique no gráfico log-log.
- Para obter erro $< 10^{-4}$: Monte Carlo precisa de quantos pontos? Compare com Gauss-Legendre. Para qual dimensão d o Monte Carlo começa a superar Gauss-Legendre em eficiência?
- Implemente para $\int_0^1 \int_0^1 e^{-(x^2+y^2)} \, dx \, dy$ (2D). Gauss-Legendre ainda funciona bem? Monte Carlo escala melhor?

Q6.3. Gradiente Numérico em Machine Learning. Em treinamento de redes neurais, o *gradient check* verifica a implementação do backpropagation comparando com diferenças finitas:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta + \varepsilon e_i) - L(\theta - \varepsilon e_i)}{2\varepsilon}.$$

- Para a função de perda $L(\theta) = \sum_{k=1}^{20} (\theta^T x_k - y_k)^2$ com dados sintéticos ($x_k \in \mathbb{R}^3$, $\theta \in \mathbb{R}^3$), calcule o gradiente analítico $\nabla_{\theta} L = 2X^T(X\theta - y)$.

- (b) Calcule o gradiente numérico por diferenças centrais com $\varepsilon = 10^{-5}$.
- (c) Calcule a diferença relativa $\|\nabla L_{analitico} - \nabla L_{numerico}\|_2 / \|\nabla L_{analitico}\|_2$. Se for $< 10^{-5}$, o gradiente está correto.
- (d) Por que não se usa ε muito pequeno no gradient check? Conecte com a análise do passo ótimo da Seção 1.

Entrega e Critérios de Avaliação

O que entregar

Formato de entrega (via Moodle, antes do prazo):

1. **Relatório PDF** com todas as respostas, tabelas preenchidas, gráficos gerados e análises escritas.
2. **Arquivos Python** organizados: `diferencas.py`, `integracao.py` e `main.py` (script que reproduz todos os resultados do relatório).
3. Cada gráfico deve ter: título, eixos rotulados com unidades, legenda e escala adequada.

Seção	Questões	Valor
1. Diferenças finitas	Q1.1–Q1.5	2,5 pts
2. Newton-Cotes	Q2.1–Q2.5	2,5 pts
3. Quadratura Gaussiana	Q3.1–Q3.3	1,5 pts
4. Análise comparativa	Q4.1–Q4.2	1,0 pt
5. Projeto integrador	Q5.1–Q5.3	2,5 pts
Total obrigatório		10,0 pts
6. Desafio (extra)	Q6.1–Q6.3	até 2,0 pts

Critérios de qualidade:

- Código limpo, comentado e executável sem erros;
- Análises escritas com rigor: não basta dizer “convergiu mais rápido” — é preciso **quantificar** (fator de redução do erro, comparação de ordens, número de avaliações de f) e **justificar** com a teoria;
- Gráficos com qualidade profissional (escala logarítmica onde relevante, retas de referência de inclinação -1 , -2 , -4);
- Conexão explícita entre resultados empíricos e a teoria apresentada em aula;
- Ênfase na **análise comparativa**: o que distingue cada método, em que cenário cada um é preferível.

A. Esqueleto do Script Principal

```

1  """
2  main.py
3  Plano de Investiga o      Diferencia o e Integra o
    Num ricas
4  Disciplina: C lculo Num rico      UNIFAL-MG
5  """
6  import numpy as np
7  import matplotlib.pyplot as plt
8  from diferencas import (df_progressiva, df_regressiva,
    df_central,
9
    df_prog3, df_retro3, d2f_central,
    derivada_tabela)
10 from integracao import (ponto_medio, trapezios, simpson13,
11    trapezios_tabela, gauss_legendre)
12
13 np.random.seed(42)
14 plt.rcParams.update({'font.size': 11, 'figure.dpi': 120})
15
16 #          Se o 1.1      F rmulas de 2 pontos

17 print("=" * 55)
18 print("SE O 1.1      F rmulas de dois pontos")
19 print("=" * 55)
20 f = np.sin
21 x0 = np.pi / 3
22 exato = np.cos(x0)
23 for h in [0.1, 0.01]:
24     prog = df_progressiva(f, x0, h)
25     regr = df_regressiva(f, x0, h)
26     cent = df_central(f, x0, h)
27     print(f"\nh = {h}")
28     print(f"    Progressiva: {prog:.8f} | erro = {abs(prog-exato)
    :.2e}")
29     print(f"    Regressiva: {regr:.8f} | erro = {abs(regr-exato)
    :.2e}")
30     print(f"    Central: {cent:.8f} | erro = {abs(cent-exato)
    :.2e}")
31
32 #          Se o 1.3      Passo timo

33 g = np.exp
34 hs = np.logspace(0, -16, 200)
35 exato_g = np.exp(1.0)
36 erros_c = np.abs(df_central(g, 1.0, hs) - exato_g)
37 erros_p = np.abs(df_progressiva(g, 1.0, hs) - exato_g)
38
39 plt.figure(figsize=(8, 4))

```

```

40 plt.loglog(hs, erros_c, 'b-', lw=1.5, label='Central  $O(h^2)$ ')
41 plt.loglog(hs, erros_p, 'r--', lw=1.5, label='Progressiva  $O(h)$ 
    ')
42 plt.axvline(1e-5, color='b', ls=':', alpha=0.6, label='$h_{opt}$
    central')
43 plt.axvline(1e-8, color='r', ls=':', alpha=0.6, label='$h_{opt}$
    prog.')
44 plt.xlabel('$h$'); plt.ylabel('Erro absoluto')
45 plt.title('Passo timo : truncamento vs. arredondamento')
46 plt.legend(); plt.grid(True, which='both', alpha=0.3)
47 plt.tight_layout(); plt.savefig('passo_otimo.pdf')
48
49 #          TODO: Se o 2      Newton-Cotes

50 # tabela_comparativa(lambda x: 1/x, 1, 2, np.log(2),
    [4,8,16,32], [2,3,4,5,6])
51
52 #          TODO: Se o 3      Gauss-Legendre

53 # f_exp2 = lambda x: np.exp(-x**2)
54 # for n in [2, 4, 6, 8, 10]:
55 #     G = gauss_legendre(f_exp2, 0, 1, n)
56 #     print(f"GL n={n}: {G:.10f}")
57
58 #          TODO: Se o 5.1    Comprimento de curva

59 # ...
60
61 plt.show()
62 print("\nConclu do. Gr ficos salvos como PDF.")

```

Listing 4: main.py — estrutura sugerida para o relatório

B. Referências

- [1] Burden, R. L.; Faires, J. D. *Numerical Analysis*, 10^a ed. Cengage, 2015. Cap. 4.
- [2] Ruggiero, M. A. G.; Lopes, V. L. R. *Cálculo Numérico*, 2^a ed. Pearson, 1996. Cap. 7–8.
- [3] Chapra, S. C.; Canale, R. P. *Numerical Methods for Engineers*, 7^a ed. McGraw-Hill, 2015.
- [4] SciPy Integrate: <https://docs.scipy.org/doc/scipy/reference/integrate.html>
- [5] NumPy Legendre: <https://numpy.org/doc/stable/reference/generated/numpy.polynomial.legendre.leggauss.html>